Video Transcript

Video 3: Exploring Threads (06:23)

We've seen that DASK allows us to run tasks in parallel. So, we looked at two functions: increment and add. And we saw that increment could be executed in parallel while add had to be executed sequentially. So, DASK comes up with a task graph, and then, that's passed to a scheduler. But, how is this executed? Here, it says that it scales up and it can run on thousands of cores. Let's take a look at a modern machine. So, here's the chipset, and here's an example of a four-core machine. So, this has four CPUs, if you like, central processing units, each of these can execute instructions. Each core can be passed separate instructions. Each core has access to its own cache, L1, which is very fast. Just a few clock ticks, three or four clock ticks, to access L1 cache. L2 cache is typically 14 or so clock ticks, and then L3 cache can be 60 or 70 clock ticks, and RAM can be hundreds or thousands of clock ticks.

So, if we want things to run very quickly, we need to think about how they're executing, how much cache they're using, and how many cores we have on the machine that can execute. Now, we talk about threads, and one way to think about it is that a core operates on a set of instructions that are passed to it on a thread. Now, often these instructions might be to go out and read a file, and that may take millions of clock ticks just to go out onto the web and retrieve a file. So, during that time the core can execute another thread. So, typically processes have multiple threads. So, here I'm showing process one has three threads. Now, threads can also have priority. If, for example, they belong to the operating system, they're going to have high priority, whereas, some of our tasks have low priority. So, threads can execute alongside each other, but they may have different priorities. And the time slicing that each thread gets will depend on its priority.

So, here I'm showing different threads being passed into. Here, we've got the four cores. Let's go and take a look at some code. So, here I've got a program, it's called threads.py. We've got a function that's being called and that function sleeps. I'm going to have it sleep for three seconds, and let's spin up five threads. So here, this loop creates a thread, appends it to our list of threads, starts that thread and then when it returns, it actually joins it back into the main thread. So, let's

© MIT xPRO® 2024. All rights reserved.

see this running. And if we look here on the right, we'll see these threads, see them running, then they're gone. Let's have them sleep a little longer. Let's have them sleep for 10, so we can see them.

So, we're executing. And now, the threads don't start with Thread-1, but we've got five of them, and then we'll see them finishing. And you'll see over here, we had zero done, then four was done, then one, then two, then three. So, although we launched them in an order, they don't necessarily return in that order. They get arbitrary amounts of time slices that the operating system decides on, and they may finish in a different order. So, you've got to be careful when you're launching threads; don't expect them to come back with their results in the same order that you launch them. We typically talk about a scatter-gather strategy. You scatter your computations out to these threads and then you gather them together to synchronize them again.

Let's take a quick look at, on my computer here, my activity app. Here, we can see this is CPU time. Here, you can see the number of threads. So, the operating system, my kernel, has 231 threads; MySQL, 38 threads. The code we just ran, 30 threads. So typically, we literally have thousands of threads running on our machine. So, why don't you take this code and why don't you spin up more threads by changing this number? And perhaps you want to have them sleep for different amounts of time. So, there's a parameter x is passed into test, and this is i being passed in. So, depending on which thread it is, we can have it sleep for a different amount of time. Okay, bye for now.

© MIT xPRO® 2024. All rights reserved.